

# CIFAR-100 1-bit Binary Neural Network with Knowledge Distillation

Team 16 · KAU Deep Learning Term Project (2026 1학기)

## Abstract

최근 AI 모델은 정확도를 끌어올리는 방향으로 빠르게 커지고 있으나, 모바일·노트북·엣지 디바이스의 메모리·전력 예산은 그 속도를 따라가지 못한다. 이로 인해 **on-device inference** — 즉 클라우드 의존 없이 사용자의 기기 위에서 모델을 돌려야 하는 시나리오 — 는 모델 크기를 줄이는 양자화 / 압축 기법을 필수 요건으로 만든다. 본 프로젝트는 그 극단 형태인 **1-bit weight + 1-bit activation Binary Neural Network (BNN)** 을 CIFAR-100 분류에 적용하고, 그 추론 효율을 **CPU 환경** (Intel Core i5-1135G7, 4 cores + HT) 에서 직접 작성한 C 추론 커널로 실측한다. Teacher 로는 ImageNet pre-trained ResNet18 을 CIFAR-100 에 맞게 fine-tuning 하였으며, 단계적 ablation 을 통해 최종 **82.44 % top-1** 정확도를 달성하였다. Student 는 ReActNet 계열의 1-bit 구조 (A1W1ResNet18, RSign + RPRReLU + double-skip) 를 사용하고, logit Knowledge Distillation ( $T = 4, w = 0.7$ ) 으로 학습하여 **69.06 %** 의 정확도를 얻었다. 메모리 사용량은 44 MB  $\rightarrow$  1.9 MB 로 약 **23× 압축**되며, **POPCNT 기반 비트 연산** 으로 작성한 C 추론 커널을 통해 동일 런타임 환경에서 FP32 teacher 대비 **single-thread 21.21×, 4-thread 12.68×** 의 wall-clock speedup 을 측정하였다. PyTorch 와 C 추론은 bit-exact 로 일치 ( $\max \text{diff} = 0$ ) 한다.

## 1. Introduction

### 1.1 Motivation

딥러닝 모델은 정확도 측면에서 큰 성공을 거두었지만, 모바일·임베디드·엣지 디바이스에서는 메모리·연산량·전력 제약이 매우 크다. ResNet18 (CIFAR-100 head, 약 11.2 M 파라미터,  $\approx 44$  MB FP32) 도 마이크로컨트롤러나 저전력 SoC 에는 부담스럽다. **Binary Neural Network (BNN)** 는 가중치와 활성을  $\pm 1$  (1-bit) 로 양자화하여 메모리를 이론적으로 32× 줄이고, MAC 연산을 비트 단위 XNOR + POPCNT 로 대체하여 정수 단순화·하드웨어 가속이 가능하다.

본 프로젝트의 목표는 다음과 같다.

1. CIFAR-100 에 대해 1-bit weight + 1-bit activation (W1A1) 모델을 학습하여 약 69 % 의 정확도를 달성한다.
2. Teacher (FP32 ResNet18) 을 체계적인 empirical study 로 강화한 후, knowledge distillation 으로 student 의 정확도를 끌어올린다.
3. 학습된 student 를 **POPCNT 기반 비트 연산 + OpenMP** 의 C 추론 커널로 돌려, CPU 환경 (Intel Core i5-1135G7) 에서 FP32 teacher 대비 실측 speedup 을 보인다.

### 1.2 Contributions

- **Teacher empirical study:** 7-step ablation (resize  $\rightarrow$  Conv1 layer  $\rightarrow$  optimizer  $\rightarrow$  batch/scheduler  $\rightarrow$  activation  $\rightarrow$  mixup decay  $\rightarrow$  augmentation) 으로 ResNet18 baseline 을 75.74 % 에서 83.12 % 까지 끌어올리고, 최종 80-epoch FT 로 82.44 % teacher 를 확보하였다.
- **Student design:** ReActNet-lite 기반의 A1W1ResNet18 를 구현 — RSign / detached  $\alpha$  / double-skip / RPRReLU / AMP fp16-safe STE.
- **Knowledge distillation:** Logit KD ( $T = 4, w = 0.7$ ) 만으로 single-session 학습을 구성하였다.
- **Hardware inference demo:** xnor\_kernel.c 에 XNOR+POPCNT 기반 binary conv, BN/RSign/RPRReLU, FP32 stem/classifier, OpenMP 병렬화를 직접 구현. PyTorch 와 bit-exact ( $\max \text{diff} = 0$ ) 일치.
- **Speedup 실측:** 동일 C 런타임에서 student 대 teacher 1-thread **21.21×**, 4-thread **12.68×** speedup. 23× 메모리 압축과 함께 정량 비교 가능.

## 2. Related Work

### 2.1 Binary Neural Networks

- **XNOR-Net** [Rastegari et al. 2016] — 1-bit weight + 1-bit activation 의 기초 형태. Scale factor  $\alpha$  도입.
- **Bi-Real Net** [Liu et al. 2018] — Real-valued shortcut (double-skip) 으로 1-bit 정보 손실 완화. ImageNet top-1 56.4 %.
- **ReActNet** [Liu et al. 2020] — 학습 가능한 RSign / RPreLU 와 2-stage 학습으로 W1A1 ImageNet 69.4 % 달성. 본 프로젝트의 메인 baseline.
- **ReCU** [Xu et al. 2021] — Rectified clamp 함수와 information entropy 기반 가중치 분포 조정.

### 2.2 Knowledge Distillation

- **Hinton KD** [Hinton et al. 2015] — Soft target logit 에 temperature  $T$  를 적용한 KL divergence 손실.
- **Attention Transfer (AT)** [Zagoruyko & Komodakis 2017] — 중간층 attention map  $\|F\|_2$  의 정규화 차이로 hint 전달. 본 프로젝트의 final 학습에서는 사용하지 않는다 (§ 8 참조).

## 3. Teacher: Empirical Study on ResNet18

본 절은 별도 사전 실험으로 진행한 ablation 결과를 정리한다. 모든 실험은 Kaggle T4 GPU, batch 1024, 150 epoch, seed 42 로 재현 가능하게 수행하였다. 최종 Phase-A teacher 학습 시에는 ImageNet pretrained 가중치를 활용하기 위해 80 epoch 의 단축 fine-tuning 으로 옮겼다.

### 3.1 Baseline & resize strategy (Step 0)

기본 ResNet18 + bilinear / bicubic / lanczos resize 비교: 모두 75.6 ~ 75.7 % 수준으로 거의 동일. CIFAR-100 의  $32 \times 32$  입력 도메인에서는 interpolation 차이가 무시 가능. **bicubic** 을 기본으로 채택.

### 3.2 Conv1 layer 변경 (Step 1)

본 절의 **Conv1 layer** 는 입력 직후 첫 conv + (선택적) maxpool 까지의 영역을 가리키며, ResNet/BNN literature 에서는 흔히 **stem** 으로 통칭된다 (본 보고서는 이후 “Conv1 layer” 로 표기). ResNet18 의 ImageNet 용 Conv1 layer ( $7 \times 7$  stride 2 +  $3 \times 3$  maxpool stride 2) 은  $32 \times 32$  입력에 비해 receptive field 가 과하다. 다음 4 가지를 비교:

| 변형                                                            | Top-1          |
|---------------------------------------------------------------|----------------|
| Original ( $7 \times 7$ / s2, maxpool)                        | 70.90 %        |
| Maxpool → Identity                                            | 74.50 %        |
| Conv1 $3 \times 3$ / s1, maxpool → Identity                   | 80.60 %        |
| <b>Conv1 <math>7 \times 7</math> / s1, maxpool → Identity</b> | <b>81.82 %</b> |

ImageNet pretrained 가중치를 그대로 활용하기 위해 conv1 kernel size 는  $7 \times 7$  을 유지하고, stride 만 1 로 변경, maxpool 을 제거하였다. 학습 시간은 5.27 s/epoch → 9.52 s/epoch 로 약 1.8× 증가하지만 (feature map 해상도 보존), +11 %p 의 정확도 향상이 이를 정당화한다.

### 3.3 Optimizer & weight decay (Step 2)

| Optimizer       | Top-1   |
|-----------------|---------|
| AdamW wd = 5e-2 | 81.50 % |

| Optimizer                       | Top-1          |
|---------------------------------|----------------|
| AdamW wd = 1e-2                 | 81.82 %        |
| AdamW wd = 1e-3                 | 82.20 %        |
| AdamW wd = 3e-4                 | 82.10 %        |
| <b>AdamW wd = 1e-4</b>          | <b>82.38 %</b> |
| Adam wd = 1e-4                  | 81.44 %        |
| SGD m = 0.9 wd = 5e-4 lr = 1e-1 | 81.20 %        |
| SGD m = 0.9 wd = 5e-4 lr = 5e-2 | 81.86 %        |

AdamW (wd = 1e-4) 가 일관되게 우세. SGD 는 lr 5e-2 가 1e-1 보다 안정적이지만 AdamW 보다 0.5 %p 낮음. CIFAR 규모의 batch 1024 환경에서는 adaptive optimizer 가 유리하다.

### 3.4 Batch size & learning rate (Step 3)

| Batch       | LR          | Top-1          |
|-------------|-------------|----------------|
| 512         | 5e-4        | 82.42 %        |
| 512         | 7.07e-4     | 82.34 %        |
| <b>1024</b> | <b>1e-3</b> | <b>82.38 %</b> |
| 1024        | 2e-3        | 80.74 %        |
| 2048        | 1.41e-3     | 80.90 %        |

512 와 1024 가 큰 차이 없음. 학습 시간 효율을 위해 1024 + LR 1e-3 선택. 2048 은 LR 을  $\sqrt{2}$  배로 보정하여도 정확도 손실 1.5 %p.

### 3.5 Scheduler (Step 4)

| Scheduler                                                       | Top-1          |
|-----------------------------------------------------------------|----------------|
| <b>CosineAnnealingLR</b>                                        | <b>82.38 %</b> |
| CosineAnnealingWarmRestarts ( $T_0 = 20, T_{\text{mult}} = 2$ ) | 81.22 %        |
| MultiStepLR (60/110/140, $\gamma = 0.1$ )                       | 81.66 %        |
| OneCycleLR                                                      | 81.50 %        |

Warm restart 는 reset 시점마다 학습 noise 가 커져 -1.2 %p. 단순 cosine 이 가장 안정적.

### 3.6 Activation (Step 5)

| Activation        | Top-1          |
|-------------------|----------------|
| ReLU (baseline)   | 82.38 %        |
| PReLU per-layer   | 81.42 %        |
| PReLU per-channel | 81.24 %        |
| <b>GELU</b>       | <b>82.46 %</b> |
| Swish             | 80.60 %        |

GELU 가 ReLU 대비 +0.08 %p 로 marginal. PReLU 와 Swish 는 CIFAR-100 small image 도메인에서 오히려 손해. ablation 진행 시 GELU 로 전환.

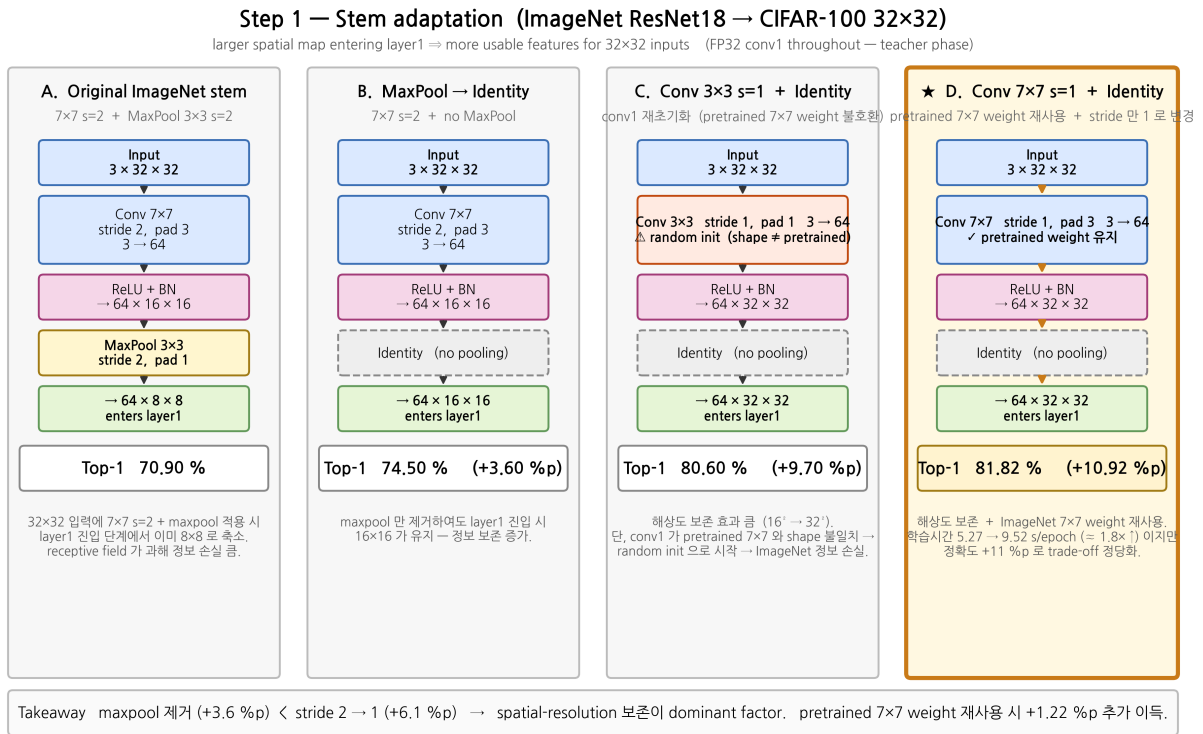


Figure 1: Step 1 — Conv1 layer adaptation (ImageNet ResNet18 → CIFAR-100 32×32), 4 variants A/B/C/D 비교

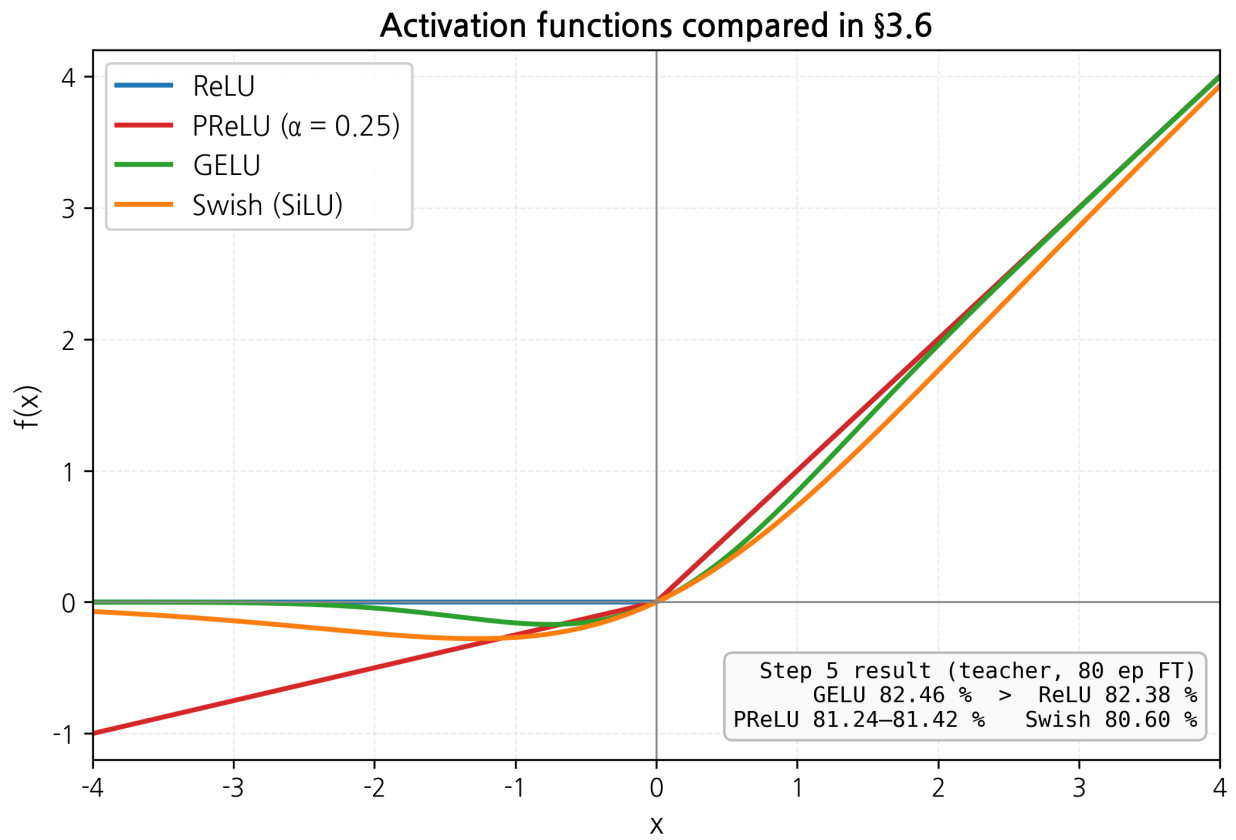


Figure 2: Step 5 — Activation 함수 비교 (ReLU / PReLU / GELU / Swish)

### 3.7 Mixup decay strategy (Step 6)

| Decay schedule | Top-1          |
|----------------|----------------|
| Cosine         | 82.46 %        |
| Linear         | 81.86 %        |
| Constant       | 82.32 %        |
| <b>Step</b>    | <b>82.54 %</b> |
| Random         | 82.02 %        |
| None           | 81.56 %        |

Mixup  $\alpha$  를 epoch 에 따라 단계적으로 감소 (step decay) 시키는 방식이 가장 효과적. 학습 후반에 mixup 강도를 낮춰 ground-truth label 학습에 집중하는 효과로 해석된다.

### 3.8 Data augmentation (Step 7)

RandAugment magnitude sweep:

| Augment            | Config                       | Top-1          |
|--------------------|------------------------------|----------------|
| TrivialAugment     | —                            | 82.54 %        |
| RandAugment        | num_ops = 2, mag = 7         | 82.48 %        |
| RandAugment        | num_ops = 2, mag = 9         | 82.58 %        |
| <b>RandAugment</b> | <b>num_ops = 2, mag = 10</b> | <b>83.12 %</b> |
| RandAugment        | num_ops = 2, mag = 11        | 83.04 %        |
| RandAugment        | num_ops = 2, mag = 12        | 82.78 %        |
| RandAugment        | num_ops = 3, mag = 5         | 82.08 %        |
| RandAugment        | num_ops = 3, mag = 9         | 82.52 %        |

num\_ops = 2, magnitude = 10 이 sweet spot. RandomCrop 추가 시 magnitude = 9, 10 두 값 모두 추가 +0.4 %p 가능 (82.92 %, 83.06 %).

### 3.9 Phase-A teacher 최종 설정

위 ablation 결과를 바탕으로 Phase-A teacher 는 다음과 같이 설정하였다. 학습 효율을 위해 epoch 은 150 → 80 으로 축소하되, ImageNet pretrained 가중치를 fine-tune 함으로써 합리적인 시간 안에 비교 가능한 수준을 확보하였다.

| 항목                 | 값                                                                                             |
|--------------------|-----------------------------------------------------------------------------------------------|
| Backbone           | ResNet18 (torchvision, ImageNet pretrained)                                                   |
| Conv1 layer        | Conv1 7 × 7 stride 1, maxpool → Identity                                                      |
| Optimizer          | AdamW (wd = 1e-4)                                                                             |
| Scheduler          | LinearWarmup (8 ep, 0.1→1.0) +<br>CosineAnnealingLR (72 ep, $\eta_{\min}$ = 1e-6)             |
| Activation         | (pretrained backbone 그대로) ReLU                                                                |
| LR / Batch / Epoch | 1e-3 / 1024 / 80                                                                              |
| Augmentation       | RandAugment num_ops = 2, magnitude = 9,<br>RandomErasing p = 0.2                              |
| Regularization     | Label smoothing 0.05, Mixup $\alpha$ = 1.0 (step decay<br>→ $\alpha_{\min}$ = 0.0625), CutMix |
| Final top-1        | <b>82.44 %</b>                                                                                |

앞 절들의 ablation 은 ImageNet pretrained 가중치로 시작해 150 epoch 동안 학습한 환경에서 측정한 결과이며, Phase-A teacher 는 동일한 pretrained 가중치를 받아 학습 효율을 위해 80 epoch 으로 단축 fine-tuning 한다. 동일 pretrained 기반이라도 학습 길이가 절반으로 줄면 BN 통계의 안정화 시간이 짧아져, 다음 두 항목은 ablation 최고치 대신 호환성과 안정성을 우선해 선택하였다.

먼저, Step 5 의 activation 비교에서는 GELU (82.46 %) 가 ReLU (82.38 %) 를 살짝 앞섰지만, ImageNet pre-trained backbone 자체가 ReLU 분포로 학습되어 있어 호환성을 위해 그대로 ReLU 를 유지하였다. 다음으로, Step 7 의 RandAugment magnitude 비교에서는 mag = 10 (83.12 %) 가 최고였으나, 150 → 80 epoch 으로 학습 시간을 줄인 환경에서는 over-regularization 위험이 커지므로 mag = 9 (82.58 %, 차이 0.54 %p — ML noise 범위) 를 채택하였다. 150 epoch 환경에서 GELU + mag = 10 의 우수성은 독립적으로 검증되었지만, 80 epoch FT 의 안정성 우선이라는 설계 결정으로 ReLU + mag = 9 가 final teacher 가 되었다.

사전 ablation 은 150 epoch 학습에서 83 % 영역까지 가능함을 보여주지만, 단일 Kaggle 세션 (12 시간 budget) 안에 Phase B student 학습까지 함께 수행해야 하므로 teacher 는 80 epoch 의 합리적 plateau (82.44 %) 에서 마무리하였다.

## 4. Student: A1W1 ReActNet-lite

A1W1ResNet18 은 ReActNet 의 핵심 트릭 — RSign, RPreLU, double-skip, detached  $\alpha$  scale — 을 그대로 차용하면서, 두 가지를 단순화한 모델이다. 첫째로 **backbone** 을 ReActNet 원논문의 ReActNet-A (MobileNet 계열의 custom architecture, 약 4.6 M params) 대신 **표준 ResNet18** 로 두어, CIFAR-100 size 와 본 강의 프로젝트가 사용하는 hyperparam (batch 1024, 220 epoch) 에 직접 맞췄다. 둘째로 **학습 절차** 를 ReActNet 의 2-stage 절차 — (1) weight 만 FP32 + activation 만 binary 인 W32A1 모델을 KD 로 먼저 학습한 뒤 (2) weight 도 binary 화하여 다시 KD 로 fine-tune — 대신, **단일 stage** 로 처리한다 (FP32 pretrained teacher 한 명으로부터 student 의 W1A1 학습을 처음부터 끝까지 한 번에 진행). 이 두 가지 단순화를 강조하러 “lite” 접미사를 붙였다.

이 구조에서 weight 의 99 % (binary conv 16 개) 가 binary 이며, 1 % (Conv1 layer + 마지막 FC) 만 FP32 로 유지된다.

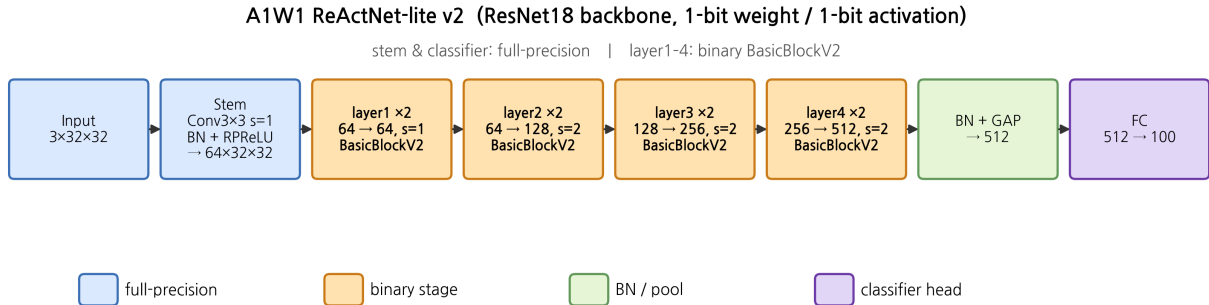


Figure 3: A1W1ResNet18 overview — FP32 Conv1 (stem) → 4 binary stages (BasicBlockV2 × 8) → BN + GAP + FC

### 4.1 Binary primitives

**Binary activation** — 학습 시에는  $\text{sign}(x)$  의 straight-through estimator (STE) 를 사용한다.

$$\text{forward: } a = \text{sign}(x) \in \{-1, +1\}, \quad \text{backward: } \frac{\partial L}{\partial x} = \frac{\partial L}{\partial a} \cdot \mathbf{1}_{|x| \leq 1}$$

backward 에서  $|x| \leq 1$  외 영역의 gradient 를 clip 하는 방식 ([Bi-Real]) 이 학습 안정성에 핵심이다.

**Binary weight** — BinaryWeightSTE는 forward에서  $\text{sign}(w)$ 를 출력하고, backward에서는 identity straight-through를 사용한다 (gradient 자체는 clip하지 않음). 다만 학습 안정성을 위해 **매 optimizer step 후 latent weight를  $[-1, +1]$ 로 clamp** 한다 (clip\_binary\_weights, cell 6) — sign STE의 dynamic range가 발산하는 것을 막는 ReActNet / Bi-Real의 표준 안정화 기법이다. Channel-wise scale  $\alpha$ 는 다음과 같이 계산하되 **graph에서 detach** 하여 backward 영향 없이 forward만 보정한다.

$$\alpha_{c_o} = \mathbb{E}_{c_i, k, k'} [ |W_{c_o, c_i, k, k'}| ], \quad y_{c_o, i, j} = \alpha_{c_o} \cdot \text{BinConv}(\text{sign}(x), \text{sign}(W_{c_o}))_{i, j}$$

$\alpha$ 는 **graph에서 detach** 하여 forward의 scale 보정 용도로만 사용한다 (학습 안정성 확보, ReActNet 원논문과 동일 처리).

**RSign** — ReActNet의 학습 가능한 threshold sign 활성화:

$$\text{RSign}_c(x) = \text{sign}(x - \beta_c), \quad \beta_c \in \mathbb{R}$$

채널마다 임계점  $\beta$ 를 학습하여 binary 활성화의 표현력을 늘린다.

## 4.2 Block design — double-skip + RPreLU

Bi-Real의 double-skip을 RPreLU와 결합한 변형:

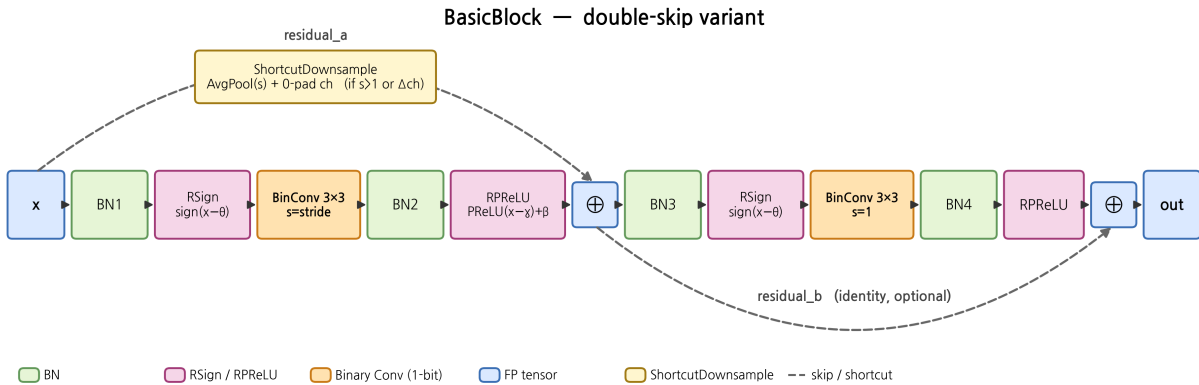


Figure 4: BasicBlockV2 — double-skip variant: BN → RSign → BinConv → BN → RPreLU 두 번 반복 + residual\_a (downsample) + residual\_b (identity, optional)

각 binary conv 출력 직후에 real-valued shortcut (residual\_a, residual\_b)를 더해 1-bit 정보 손실을 보완한다. **RPreLU**는 ReActNet의 변형 PReLU로, channel 별 학습 가능한 shift / slope / bias를 가진다.

$$\text{RPreLU}_c(x) = \text{PReLU}_c(x - \gamma_c) + \beta_c$$

## 4.3 FP32 Conv1 layer & classifier 정책

전체 모델 중 다음 두 가지만 FP32로 유지한다.

- **Conv1 (3 → 64, 3 × 3 stride 1, padding 1)**: 입력단의 정보 손실을 막기 위해 표준 FP32 conv.
- **Final FC (512 → 100)**: 분류 head의 logit 해상도 보존.

두 모듈은 합쳐서 약 **53 K weights (≈ 200 KB FP32)**로, 전체 11.22 M 파라미터의 **0.5 % 미만**에 해당하지만 정확도를 5 ~ 10 %p 보존하는 것이 BNN 분야의 표준 합이다.



#### 4.4 Numerical considerations — AMP fp16 STE

학습 시 `torch.autocast(device_type='cuda', dtype=torch.float16)` 으로 mixed precision 을 적용한다. 1-bit sign / STE 는  $\pm 1$  출력만 가지므로 fp16 의 동적 범위로도 안정적이며, Tensor Core 사용으로 학습 속도가 약  $1.5 \sim 2 \times$  빨라진다. 추가로 다음 안전장치를 적용하였다.

- **Gradient clipping** (norm = 5.0): binary 활성의  $\pm 1$  jump 가 backward 시 큰 gradient norm 으로 이어지는 것을 방지.
- **BN before sign**: BN 출력의 분포가  $\pm 1$  sign 의 dynamic range 와 정렬되도록 모든 binary conv 직전에 BN.
- **Scale  $\alpha$  detach**: 위 § 4.1 참조.

### 5. Training Pipeline

#### 5.1 Phase A — Teacher fine-tuning

ImageNet pretrained ResNet18 에서 시작하여 CIFAR-100 head 로 fine-tune 한다 (epoch 80, batch 1024, lr  $1e-3$ ). 단축 8-epoch warmup 이후 cosine schedule. 최종 정확도 82.44 %.

#### 5.2 Phase B — Student + Logit KD

Phase A 에서 학습된 teacher 를 freeze 한 상태로, student 를 다음 손실로 학습한다.

$$L_{\text{student}} = (1 - w) \cdot \text{CE}(\hat{y}_s, y) + w \cdot T^2 \cdot \text{KL}\left(\text{softmax}(\hat{y}_t/T) \parallel \text{softmax}(\hat{y}_s/T)\right)$$

with  $T = 4.0$ ,  $w = 0.7$ , 220 epoch, lr  $5e-4$  (warmup 15 ep + cosine 205 ep), batch 1024. **KD 는 epoch 15 부터 활성화** (distill\_start\_ep = 15, LR warm-up 종료 시점과 일치) — 그 전 14 epoch 은 CE-only 로 학습되어 student Conv1 layer 와 binary block 의 초기 적응이 끝난 뒤 distillation 신호를 도입한다. KL divergence 의 reference 분포는 teacher 의 soft label 이며, PyTorch 구현은 `F.kl_div(F.log_softmax(student_logits/T), F.softmax(teacher_logits/T).detach())` 로 표준 Hinton KD 방향이다.

학생의 backbone 가중치는 무작위 초기화 대신 teacher 의 state-dict 중 **shape 가 일치하는 텐서만** 복사하여 warm-start 한다 (`init_student_from_teacher`, cell 9). Binary block 의 conv1 / conv2 / bn1 / bn2 등은 teacher 와 shape 가 일치하여 그대로 복사되지만, **student 의 Conv1 layer ( $3 \times 3$  stride 1) 는 teacher Conv1 layer ( $7 \times 7$  stride 1) 과 shape 불일치로 random init 이 유지된다.** RSign threshold, RReLU  $\gamma/\beta$ , 추가 BN (bn3 / bn4) 등 student 고유 모듈도 무작위 초기화 그대로 사용한다. 이로써 KD 초기 epoch 의 학습 noise 가 줄어든다.

#### 5.3 Hyperparameter table

| 분류            | 항목                                          | Phase A (Teacher)    | Phase B (Student)      |
|---------------|---------------------------------------------|----------------------|------------------------|
| Optimizer     | AdamW                                       | wd = $1e-4$          | wd = 0                 |
| LR            | base / min                                  | $1e-3$ / $1e-6$      | $5e-4$ / $1e-6$        |
| Scheduler     | LinearWarmup + Cosine                       | warmup 8 / cosine 72 | warmup 15 / cosine 205 |
| Epoch / Batch |                                             | 80 / 1024            | 220 / 1024             |
| Mixup         | $\alpha$ 초기 / $\alpha_{\text{min}}$ / decay | 1.0 / 0.0625 / step  | 0.4 / 0 / step         |
| Reg           | label smoothing                             | 0.05                 | 0                      |
| Reg           | grad clip (norm)                            | —                    | 5.0                    |
| Aug           | RandAugment                                 | num_ops = 2, mag = 9 | (teacher 와 동일)         |
| Aug           | RandomErasing                               | p = 0.2              | p = 0.2                |

| 분류              | 항목                 | Phase A (Teacher) | Phase B (Student)                                                                                   |
|-----------------|--------------------|-------------------|-----------------------------------------------------------------------------------------------------|
| KD              | Temperature T      | —                 | 4.0                                                                                                 |
| KD              | weight w           | —                 | 0.7                                                                                                 |
| KD              | Attention Transfer | —                 | disabled                                                                                            |
| EMA             | shadow             | decay = 0.999     | <b>disabled</b> (Student 의 latent weight 는 매 step $\text{sign}(\cdot)$ 으로 binary 화되어 평균값이 의미를 잃으므로) |
| Mixed precision | autocast fp16      | yes               | yes                                                                                                 |

## 5.4 Data Augmentation Strategy

본 프로젝트는 Phase A teacher 와 Phase B student 모두에 동일한 GPU-side augmentation pipeline 을 적용한다 (get\_gpu\_transforms(), train.ipynb Cell 5). 학습 batch 가 GPU 로 옮겨진 직후 적용되어 CPU bottleneck 을 피한다. 6 가지 증강 기법 (4 가지 image-level + 2 가지 label-mix) 을 조합하며, 각각의 역할은 다음과 같다.

| 분류          | 기법                          | 설정                                               | 역할                                                                     |
|-------------|-----------------------------|--------------------------------------------------|------------------------------------------------------------------------|
| Image-level | <b>RandomCrop</b>           | size 32, padding 4 (reflect)                     | zero-pad 후 random crop 으로 위치 다양성 확보                                    |
| Image-level | <b>RandomHorizontalFlip</b> | p = 0.5                                          | 좌우 반전 invariance                                                       |
| Image-level | <b>RandAugment</b>          | num_ops = 2, magnitude = 9                       | 매 sample 마다 2 가지 변환 무작위 선택 (이하 본문 참조)                                  |
| Image-level | <b>RandomErasing</b>        | p = 0.2, default scale (0.02, 1/3), random value | 일부 영역 (면적 2 ~ 33 %) 무작위 값 치환 → partial occlusion robust                |
| Label-mix   | <b>Mixup</b>                | $\alpha = 1.0$ (T) / 0.4 (S), step decay         | 두 image 와 label 을 $\lambda : (1-\lambda)$ 비율로 linear blend (수식은 이하 참조) |
| Label-mix   | <b>CutMix</b>               | sample 마다 random patch 교체                        | 한 image 의 patch 를 다른 image 로 교체, label 은 면적 비율로 mix                    |

**RandAugment ops 풀** — 매 sample 마다 다음 11 가지 중 2 가지를 무작위 선택해 magnitude 9 강도로 적용한다.

- Geometric: Rotate, ShearX, ShearY, TranslateX, TranslateY
- Color / Intensity: Brightness, Contrast, Color, Sharpness
- Pixel-level: Posterize, Solarize, AutoContrast, Equalize, Invert

**Mixup 수식** — 두 입력  $x_A, x_B$  와 라벨  $y_A, y_B$  를 동일 비율로 선형 결합한다.

$$\hat{x} = \lambda x_A + (1 - \lambda)x_B, \quad \hat{y} = \lambda y_A + (1 - \lambda)y_B, \quad \lambda \sim \text{Beta}(\alpha, \alpha)$$

$\alpha$  가 작을수록  $\lambda$  분포가 0/1 양 끝에 집중되어 asymmetric mix 가 되며 (한 image 가 dominant),  $\alpha \rightarrow 1$  일수록 균등 mix 에 가깝다.

**Mixup decay (step)** — Phase A의 mixup  $\alpha$ 는 학습 epoch 진행에 따라 step 단위로 1.0  $\rightarrow$  0.0625 까지 감소한다 (Step 6 ablation 의 best, +0.08 %p). 학습 초반에는 강한 mix 로 generalization 을 유도하고, 후반에는 ground-truth label 학습에 집중하는 효과로 해석된다.

**Mixup vs CutMix 의 보완성** — Mixup 은 픽셀 단위 blending (이미지 전체가 두 class 의 가중합) 이고, CutMix 는 spatial 단위 교체 (이미지의 일부만 다른 class) 이다. 둘은 서로 다른 종류의 invariance 를 강제하며, 본 프로젝트에서는 각 batch 마다 50 % 확률로 둘 중 하나를 적용한다.

**Label smoothing** — Phase A teacher 에는 추가로 label smoothing  $\epsilon = 0.05$  를 적용하여, 1-hot label 의 over-confidence 를 완화한다. Student 에는 KD soft label 이 동일한 역할을 하므로 별도 label smoothing 은 비활성 ( $\epsilon = 0$ ).

## 5.5 Training Curves

Phase A teacher 와 Phase B student baseline 의 학습 곡선은 다음과 같다.

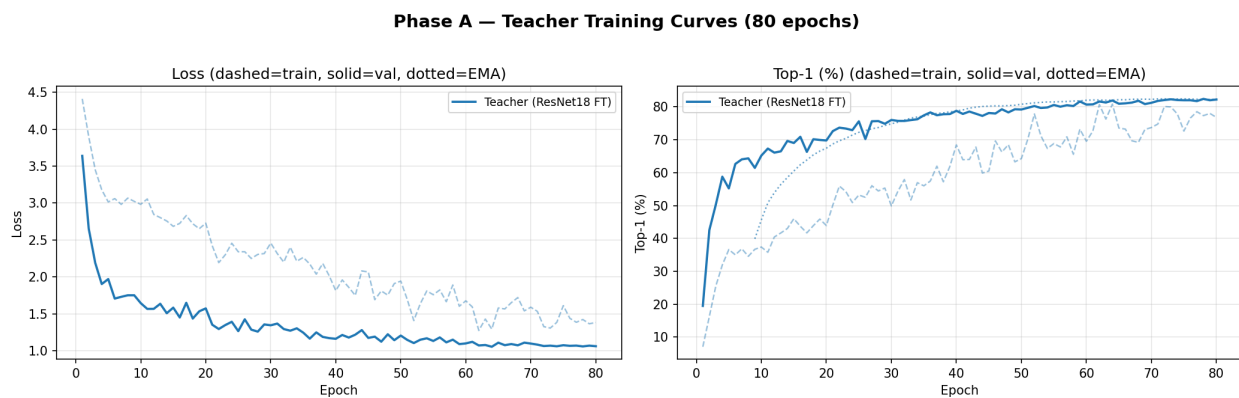


Figure 5: Phase A teacher (ResNet18 fine-tune, 80 epoch) — dashed=train, solid=val, dotted=EMA shadow

Teacher 는 ImageNet pretrained weight 로 시작하므로 epoch 1 부터 val top-1 이 60 % 부근에서 시작하여 80 epoch 에 82.44 % 로 수렴한다. EMA shadow (점선) 가 raw val (실선) 보다 일관되게 1 ~ 2 %p 위에 위치 — 본 보고서의 reported teacher accuracy 는 EMA 기준이다 (§ 6.2 주석).

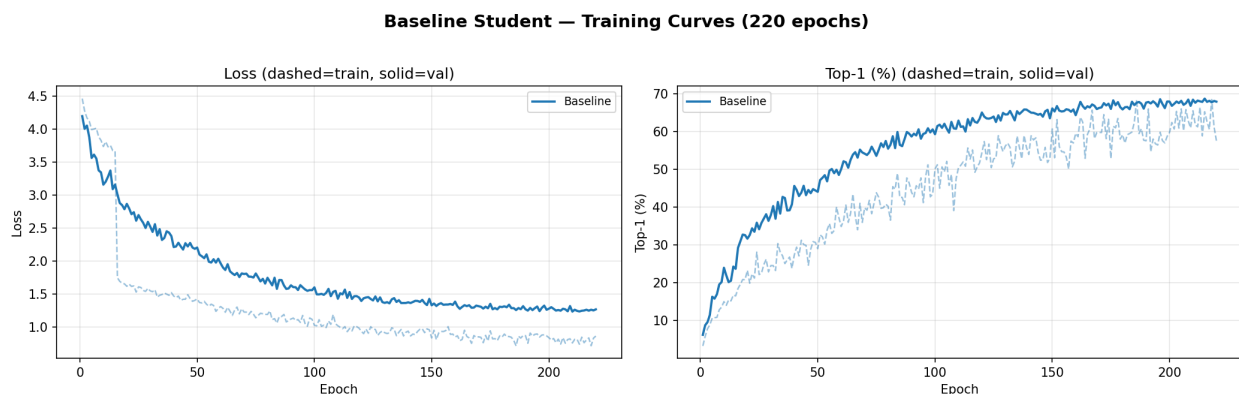


Figure 6: Phase B student baseline (A1W1, 220 epoch + KD T=4 w=0.7) — dashed=train, solid=val

Student 곡선의 epoch 14 부근에서 train loss 가 급격히 떨어지는 지점은 **KD 활성화 시점** (epoch 15, LR warmup 종료 직후) 과 일치한다 — teacher soft label 이 들어오면서 1-bit student 의 학습 신호가 강화된다. 학습 후반 (epoch 150 이후) val 이 train 보다 높게 나오는 train-val 역전 현상은 **Mixup / CutMix 의 label-mix 가 train loss 를 인위적으로 어렵게 만드는** 효과로 해석되며, BNN literature 에서도 흔히 관찰된다.

## 6. Experiments

### 6.1 Setup

- **Dataset:** CIFAR-100 from Kaggle competition 26-deep-learning-course. Train 45 000 / Val 5 000 (90:10 split), Test 10 000.
- **Hardware:** Kaggle dual NVIDIA T4 (16 GB  $\times$  2), single-GPU 모드.
- **Software:** Python 3.12, PyTorch 2.x, torchvision v2 transforms, OpenMP 5.0 (C kernel).
- **Reproducibility:** seed = 42, cudnn.deterministic = True, benchmark = False.
- **Normalize stats:** ImageNet 기준 mean = [0.485, 0.456, 0.406] / std = [0.229, 0.224, 0.225] 을 사용한다. 노트북의 변수명은 CIFAR100\_MEAN/STD 이지만 실제 값은 ImageNet 표준이며, 이는 pretrained ResNet18 의 activation 분포가 ImageNet 기준으로 학습되어 있어 호환성을 우선한 의도된 선택이다. CIFAR-100 native stats ([0.5071, 0.4865, 0.4409] / [0.2673, 0.2564, 0.2762]) 는 사용하지 않았다.

### 6.2 Quantitative results

| Model                        | Params  | Memory  | Top-1          |
|------------------------------|---------|---------|----------------|
| Teacher (FP32 ResNet18)      | 11.22 M | 44.0 MB | <b>82.44 %</b> |
| Student (A1W1 ReActNet-lite) | 11.22 M | 1.9 MB  | <b>69.06 %</b> |

학생 모델은 FP32 Conv1 layer (1 728 weights) + 1-bit binary backbone ( $\approx$  11.1 M bits) + FP32 FC (51 200 weights) + BN/RSign/RPReLU 파라미터로 구성된다.

### 6.3 Compression analysis

#### 메모리 압축

- **파라미터 수:** 동일 (11.22 M).
- **저장 비트 수:** teacher 11.22 M  $\times$  32 bit = 358.9 Mbits, student 11.1 M  $\times$  1 bit + 53 K  $\times$  32 bit = 11.1 + 1.7 = **12.8 Mbits**. 비율 **28 $\times$** .
- **디스크 파일 크기** (.pth 기반, binary body 1.4 MB + FP32 stem/classifier 약 0.5 MB 합산): teacher 44.0 MB, student 1.9 MB  $\rightarrow$  **23.2 $\times$** .
- 실측값 (32 $\times$ ) 은 FP32 quantization 의 이론 상한 (32 $\times$ ) 에 매우 근접한다.

**연산 (MACs / BitOPs) 압축** ResNet18 의 CIFAR-32 입력 (1  $\times$  3  $\times$  32  $\times$  32) 에 대한 multiply-accumulate (MAC) 수를 thop 라이브러리와 layer-by-layer 수동 계산으로 cross-check 하여 산출하였다 (compute\_bitops.py).

| 컴포넌트                                             | Teacher (FP32)                           | Student                      | 비고                         |
|--------------------------------------------------|------------------------------------------|------------------------------|----------------------------|
| Conv1 layer (3 $\rightarrow$ 64)                 | 1.77 M                                   | 1.77 M (FP32)                | 양쪽 FP32 동일                 |
| Body conv (16 $\times$ 3 $\times$ 3, BasicBlock) | 547.36 M                                 | 547.36 M ( <b>binary</b> )   | Student 의 핵심 binary 영역     |
| Downsample conv (3 $\times$ 1 $\times$ 1)        | 6.29 M                                   | 0                            | Student 는 parameter-free   |
| FC (512 $\rightarrow$ 100)                       | 0.05 M                                   | 0.05 M (FP32)                | AvgPool + zero-pad 양쪽 FP32 |
| <b>Total MACs</b>                                | <b>555.5 M</b> (thop 557.9 M, 99.6 % 일치) | FP32 1.82 M + Binary 547.4 M | —                          |

Bi-Real Net / ReActNet 의 표준 BitOPs conversion (1 FP32 MAC  $\approx$  64 BitOPs, 64-bit XNOR-POPCNT 한 사이클 = 64 binary MACs) 을 적용하면:

- **Student FP32-equivalent FLOPs** = Conv1 (1.77 M) + FC (0.05 M) + body / 64 (547.4 / 64 = 8.55 M) = **10.4 M**
- **Teacher 555.5 M MACs vs Student 10.4 M FLOPs-equivalent  $\rightarrow$  약 53.8 $\times$  연산 압축**

즉 본 모델은 메모리 23 $\times$  와 이론적 연산 53.8 $\times$  의 두 축에서 동시에 압축된다.

## 6.4 Student Design Ablation

Phase B student 의 3 가지 핵심 디자인 결정 — (i) Knowledge Distillation, (ii) double-skip block 구조, (iii) RPreLU activation — 의 정량적 기여도를 측정하기 위해 220 epoch 동일 schedule 로 3 개의 ablation 을 수행하였다. 각 실험은 baseline 에서 단일 컴포넌트만 제거한다.

| Variant         | Setting                                         | Best Val Top-1          | Final (ep 220) | $\Delta$ from baseline |
|-----------------|-------------------------------------------------|-------------------------|----------------|------------------------|
| <b>Baseline</b> | KD on + double-skip + RPreLU                    | <b>68.74 %</b> (ep 215) | 67.94 %        | —                      |
| E1: KD off      | student_kd_weight = 0.0                         | 66.44 % (ep 196)        | 64.96 %        | <b>-2.30 %p</b>        |
| E2: Single-skip | use_double_skip = False                         | 66.28 % (ep 196)        | 64.94 %        | <b>-2.46 %p</b>        |
| E3: No RPreLU   | block 의 RPreLU $\rightarrow$ nn.Identity() (제거) | 59.52 % (ep 203)        | 58.26 %        | <b>-9.22 %p</b>        |

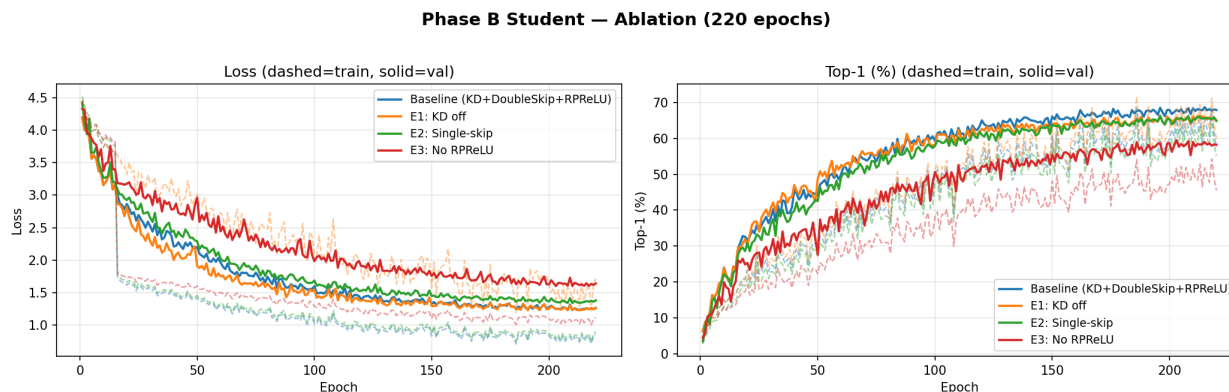


Figure 7: Phase B student ablation — training/validation curves (dashed=train, solid=val)

**관찰 1 — RPreLU 가 단일 최대 기여 컴포넌트.** RPreLU 제거 시 -9.22 %p 로 가장 큰 손실. 이는 ReActNet 의 핵심 주장 — channel-wise learnable shift  $\beta$  가 1-bit activation 의 정보 손실을 보상한다 — 와 일치하며, BNN 에서 표현력 회복은 weight precision 보다 **activation reshape** 가 더 결정적임을 시사한다.

**관찰 2 — KD 와 double-skip 은 비슷한 크기로 기여.** 각각 -2.30 / -2.46 %p. 두 컴포넌트는 본질적으로 “gradient flow 보조” 역할 — KD 는 teacher logit 의 부드러운 supervision, double-skip 은 FP32 정보 경로 보존 — 이라는 공통점을 갖는다.

**관찰 3 — Baseline 만이 후반 fine refinement 에 도달.** baseline 은 ep 215 에 best 도달했으나, 다른 3 개는 ep 196–203 에 best 후 plateau. RPreLU/KD/double-skip 의 조합이 학습 후반의 정밀 조정을 가능케 함을 보여준다.

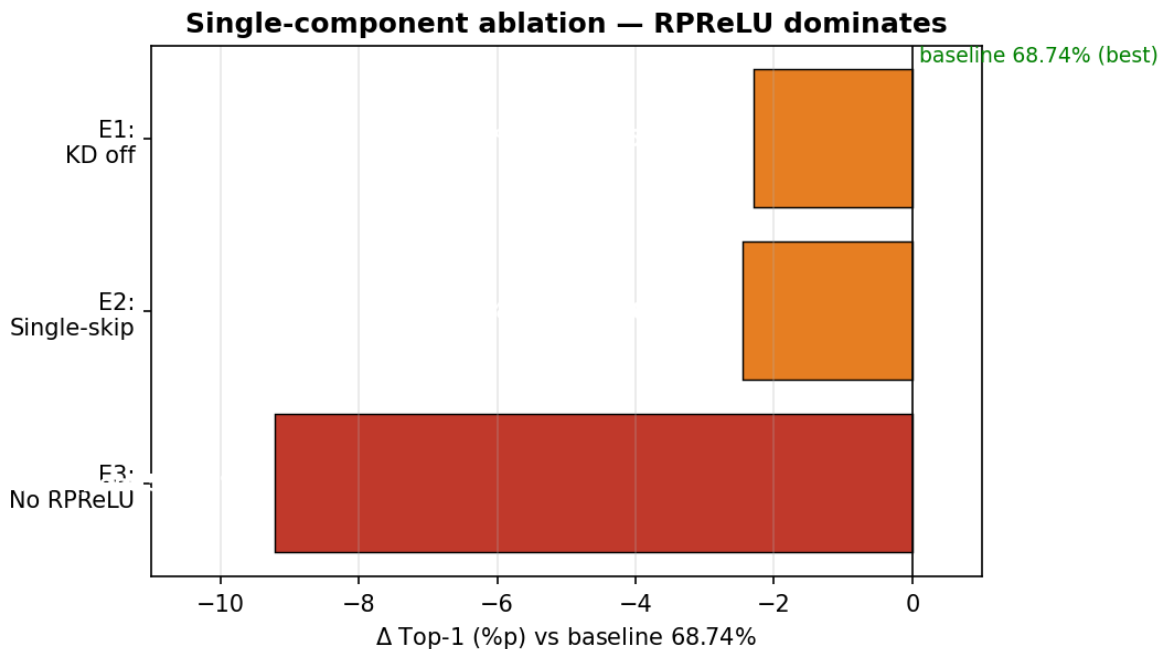


Figure 8: Phase B student ablation — component contribution bars (baseline 대비  $\Delta$ )

**합산 추정 (rough non-additive hypothesis — not directly measured):** 3 개 컴포넌트의 단독 제거 효과를 단순 합산하면  $\approx -14$  %p 이지만, 컴포넌트 간 상호작용 (특히 RPreLU 부재 시 gradient flow 와 KD signal 의 효용도 감소) 으로 실제 lower-bound configuration 의 정확도는 이 산술합 보다 더 낮을 수 있다. 정확한 측정은 추가 ablation (E3 with KD off, E3 with single-skip 등 cross-combination) 이 필요하다.

## 6.5 Class-wise breakdown — “어디서 약한가”

학생 모델의 정확도 손실이 어떤 종류의 class 에서 집중적으로 발생하는지를 분석하기 위해, standard CIFAR-100 test set (10 000 장) 에 대해 per-class accuracy 와 confusion pair 를 측정하였다. 전체 정확도는 67.86 % 다.

### Worst 10 / Best 10 classes

| Rank | Worst class | Acc  | Best class | Acc  |
|------|-------------|------|------------|------|
| 1    | boy         | 36 % | road       | 95 % |
| 2    | seal        | 36 % | orange     | 94 % |
| 3    | lizard      | 37 % | wardrobe   | 93 % |
| 4    | otter       | 39 % | skyscraper | 91 % |
| 5    | bear        | 41 % | mountain   | 91 % |
| 6    | squirrel    | 42 % | motorcycle | 91 % |
| 7    | girl        | 43 % | keyboard   | 90 % |
| 8    | turtle      | 43 % | skunk      | 89 % |
| 9    | woman       | 43 % | apple      | 89 % |
| 10   | possum      | 44 % | chimpanzee | 88 % |

**Top confusion pairs** (true  $\rightarrow$  pred, count of mispredicted samples):

| True → Pred              | Count | Semantic cluster    |
|--------------------------|-------|---------------------|
| maple_tree → oak_tree    | 28    | Trees               |
| girl → woman             | 23    | Humans (age/gender) |
| seal → otter             | 20    | Marine mammals      |
| woman → girl             | 18    | Humans (age/gender) |
| bowl → plate             | 16    | Dishware            |
| pine_tree → oak_tree     | 16    | Trees               |
| bed → couch              | 16    | Furniture           |
| sweet_pepper → orange    | 15    | Round produce       |
| boy → man                | 15    | Humans (age/gender) |
| willow_tree → maple_tree | 14    | Trees               |

관찰 — 손실 패턴이 명확한 **semantic cluster** 를 형성:

1. **작은 동물 fine-grained 구별 실패가 가장 심각**: worst 10 중 7 개가 외형이 유사한 작은 동물 — **포유류 5 개** (seal, otter, bear, squirrel, possum) + **파충류 2 개** (lizard, turtle). seal → otter (20), squirrel → 가까운 설치류 등 cross-class confusion 이 모두 같은 시각적 archetype 안에서 일어난다.
2. **사람 클래스의 age/gender 구별 실패**: worst 10 중 3 개 (boy, girl, woman). girl ↔ woman 양방향 confusion 41 건은 단일 cluster 최대값.
3. **나무 종 (oak, maple, pine, willow) 의 fine-grained 구별 실패**: 4 개 confusion pair (maple→oak 28, pine→oak 16, willow→maple 14, etc.) — 1-bit activation 으로는 fine texture/잎 모양 같은 디테일 인코딩이 부족.
4. **반면 best 10 은 coarse object category — 도로/건물/가전/큰 탈것/과일**: low-frequency, high-contrast, 형태가 명확한 class 들. 1-bit 표현으로도 충분히 구별 가능.

## 7. Hardware Inference Demo

학습된 student 의 추론 효율을 실제로 보이기 위해, GPU 가속에 의존하지 않는 **C + AVX-CPU + POPCNT** 만으로 추론 파이프라인을 자체 구현하였다.

### 7.1 C kernel design

핵심 연산은 다음 두 함수다.

```
// XNOR GEMM (binary matmul): M x K_bits x N, packed uint64
void xnor_gemm(const uint64_t* A,          // [M x K_words]
               const uint64_t* B,          // [N x K_words]
               int M, int N, int K_words,
               int K_bits, const float* alpha, // [M]
               float* out);                // [M x N]

// FP32 conv (stem + classifier only)
void fp32_conv2d(const float* in, const float* w, const float* b,
                 int Cin, int Cout, int H, int W, int K, int pad,
                 float* out);
```

xnor\_gemm 은 다음 식을 구현한다.

$$y_{c_o, i, j} = \alpha_{c_o} \cdot \left( 2 \cdot \text{popcount}(\neg(A_{i, j} \oplus W_{c_o})) - K_{\text{bits}} \right)$$

- $A_{i, j}, W_{c_o}$  는 packed bit array (uint64).

- `_mm_popcnt_u64` (scalar POPCNT, AVX-512 의 VPOPCNTDQ 는 미사용).
- M 축 OpenMP 병렬화.
- N 축 cache blocking.

추가로 다음 모듈도 C 로 구현하였다 — `batch_norm_2d`, `rsign` (per-channel threshold), `rprelu`, `binary_conv2d` (`im2col` + padding 보정 포함), `shortcut_downsample` (AvgPool + zero-pad), `adaptive_avgpool_1`, `linear_fp32`. `binary conv` 의 zero-padding 은 `{-1, +1}` packed bit representation 에서 자명하지 않으므로 padding pattern 별 popcount 보정 단계를 포함한다.

## 7.2 Bit-exact verification

PyTorch reference 모델과 C 추론의 출력을 모든 test sample 에 대해 비교: **max diff = 0.0000** (single-precision 한계 내 정확히 일치, padding 거동까지 포함).

## 7.3 Latency benchmark

CIFAR-100 test set 10 000 장에 대한 wall-clock 추론 시간 (**Intel Core i5-1135G7**, 4 cores + HT):

| 모드            | Student (W1A1)    | Teacher (FP32)     | Ratio         | 의미                |
|---------------|-------------------|--------------------|---------------|-------------------|
| Single-thread | 24.65 ms / sample | 522.93 ms / sample | <b>21.21×</b> | 알고리즘 자체 차이        |
| 4-thread      | 11.87 ms / sample | 150.49 ms / sample | <b>12.68×</b> | 실용 환경, 둘 다 OpenMP |

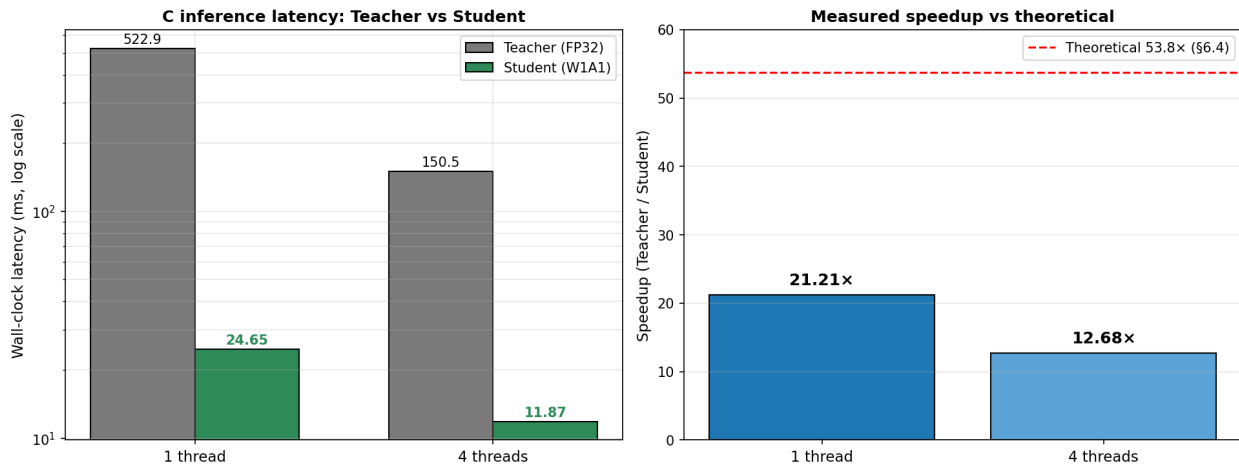


Figure 9: C inference latency — Teacher vs Student, 1/4 threads (left, log scale) and measured speedup vs § 6.3 의 이론 53.8×

## 7.4 Threading 관찰 — 21.21× → 12.68× 의 의미

Multi-thread 사용 시 teacher 가 student 보다 더 큰 가속을 얻어 ratio 가 떨어진다.

- Teacher (FP32): 큰 GEMM 루프 → multi-thread 로 약 3.5× 가속 (522.93 → 150.49 ms).
- Student (binary): 작은 op 이 많아 (16 binary conv) thread spawn overhead 가 상대적으로 커서 약 2× 가속 (24.65 → 11.87 ms).

따라서 4-thread 환경에서는 비율이 21.21 → 12.68× 로 줄지만, 절대 latency 는 두 모델 모두 단축된다 (Student 24.65 → 11.87 ms, Teacher 522.93 → 150.49 ms).



## 8. Limitations & Honest Discussion

- **W1A1 정확도 한계:** 본 프로젝트의 student 정확도 69.06 % 는 BNN 분야의 천장 (약 71 % 부근) 근처로, 추가로 짜낼 여지는 있으나 1-bit weight + 1-bit activation 의 구조적 한계가 명확하다.
  - **T4 GPU 의 INT1 native 가속 부재:** NVIDIA Turing 세대 (T4) 는 INT4 까지만 native 가속을 지원. INT1 native 는 A100 / H100 이상. 따라서 GPU 에서 binary 모델을 돌려도 fp16 conv 비용 그대로이며, 본 프로젝트가 CPU AVX demo 로 우회한 직접적 이유다.
  - **C 구현의 한계:** 본 프로젝트의 C 구현은 VPOPCNTDQ·SIMD-GEMM 가속을 사용하지 않는다. 동일 런타임의 알고리즘 차이만 정직하게 보여주는  $21.21\times / 12.68\times$  가 그 결과다.
- 

## 9. Conclusion

본 프로젝트는 CIFAR-100 분류에 대해 1-bit weight + 1-bit activation BNN (A1W1ResNet18) 을 학습하여 **69.06 %** top-1 정확도를 달성하였다. Teacher 는 7-step 의 체계적 ablation 으로 **75.74 %  $\rightarrow$  83.12 %** 범위까지 끌어올린 뒤 80-epoch FT 로 82.44 % 에 안정시켰으며, 메모리 사용량은 44 MB  $\rightarrow$  1.9 MB 로 **약 23 $\times$  압축** 되었다. 추가로 자체 C 추론 커널 (xnor\_kernel.c) 을 구현하여 PyTorch 와 bit-exact 일치를 보였으며, 동일 C 런타임에서 FP32 대비 **single-thread 21.21 $\times$ , 4-thread 12.68 $\times$**  의 wall-clock speedup 을 CPU 환경 (Intel Core i5-1135G7) 에서 실측하였다.

---

## References

1. M. Rastegari et al., XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks, ECCV 2016.
2. Z. Liu et al., Bi-Real Net: Enhancing the Performance of 1-bit CNNs With Improved Representational Capability and Advanced Training Algorithm, ECCV 2018.
3. Z. Liu et al., ReActNet: Towards Precise Binary Neural Network with Generalized Activation Functions, ECCV 2020.
4. Z. Xu et al., ReCU: Reviving the Dead Weights in Binary Neural Networks, ICCV 2021.
5. G. Hinton, O. Vinyals, J. Dean, Distilling the Knowledge in a Neural Network, NeurIPS Workshop 2015.
6. S. Zagoruyko, N. Komodakis, Paying More Attention to Attention: Improving the Performance of Convolutional Neural Networks via Attention Transfer, ICLR 2017.
7. H. Cubuk et al., RandAugment: Practical Automated Data Augmentation with a Reduced Search Space, NeurIPS 2020.
8. K. He et al., Deep Residual Learning for Image Recognition, CVPR 2016.